

Empirical Analysis of Post-Quantum Cryptographic Handshake Resilience in Lossy IoT Networks: A Comparative Study of Kyber-TLS and ECC-TLS Under Adverse Network Conditions

Maisam Raza
2023463
Ghulam Ishaq Khan Institute

Abdullah Ahtasham
2023033
Ghulam Ishaq Khan Institute

Muhammad Shaheem
2023507
Ghulam Ishaq Khan Institute

Abstract—In this paper, an experimental analysis of Post-Quantum Cryptographic (PQC) handshake resilience in the context of resource-limited IoT scenarios is performed by contrasting the handshake performance of Kyber-512 based TLS (KYBER-TLS) and traditional Elliptic Curve Cryptography TLS (ECC-TLS). Building upon the PQC-IoTNet benchmarking framework, 1,260 experimental sessions have been performed for varying packet loss rates (0–10%) and jitter magnitudes (0, 20, and 50 ms), employing the NetEm mechanism available in Linux on the MQTT/TLS protocol suite. While both handshake schemes succeeded in establishing connections in all experimental scenarios, at baseline, KYBER-TLS completed handshakes in 397.9 ms on average against 588.8 ms for ECC-TLS which is a 32.4% gap and held lower variance as network conditions worsened. Jitter Sensitivity Index values stayed close to 1.0 for both schemes, pointing to broadly similar jitter behaviour. On the whole, KYBER-TLS appears well-suited for lossy IoT deployments and presents a credible path toward post-quantum migration.

Index Terms—Post-Quantum Cryptography, Kyber, TLS, IoT Security, MQTT, Network Emulation, Handshake Latency, ECC, NetEm, Jitter Sensitivity

I. INTRODUCTION

The number of Internet of Things (IoT) devices being deployed globally has increased dramatically, with projections of up to 29 billion connected devices by 2030 [1]. IoT devices often operate in wireless mediums where packet loss and latency variation are common. Security at the communications layer of IoT devices requires cryptographic algorithms that are resistant to quantum attacks and capable of operating effectively under such impairments.

The impending threat posed by Shor’s algorithm to both Elliptic Curve Cryptography (ECC) and RSA cryptosystems has motivated NIST to develop new PQC standards [2]. Among those, CRYSTALS-Kyber (Kyber), a key encapsulation mechanism (KEM) based on lattices, was approved by NIST as the main PQC algorithm for key exchange. While various theoretical advantages have been reported, the operational performance of Kyber under adverse network conditions in resource-constrained IoT networks is insufficiently studied.

To address this issue, Almutairi et al. [3] presented a framework named PQC-IoTNet for evaluating PQC handshakes

in IoT environments. Nonetheless, their experiments were restricted to only two packet loss levels (0% and 5%), 20 trials per scenario, and absence of jitter tests. This paper intends to close the existing research gaps in the field. This work directly addresses these gaps.

Our contributions are:

- 1) A high-resolution packet loss gradient (seven levels from 0–10%) with 30 trials per condition;
- 2) Systematic jitter injection at 0, 20, and 50 ms across all conditions;
- 3) A comprehensive statistical analysis including mean, median, standard deviation, 95th percentile, and worst-case latency;
- 4) A Jitter Sensitivity Index (JSI) characterising relative susceptibility of each algorithm to latency variance; and
- 5) A complete dataset of 1,260 trials for future PQC benchmarking research.

II. BACKGROUND AND RELATED WORK

A. Post-Quantum Cryptography and Kyber

CRYSTALS-Kyber is an efficient lattice-based KEM that relies on the problem of Module Learning With Errors (MLWE) [4]. Kyber-512 aims at NIST security Level 1 (equivalent to AES-128) and delivers compact keys: an 800-byte public key and a 768-byte ciphertext. This results in considerably more compact keys compared to most PQC counterparts, making Kyber especially appealing for constrained devices. The Open Quantum Safe (OQS) Project offers production-ready implementations of Kyber in OpenSSL via the `oqs-provider` plugin [5], allowing seamless replacement of ECC key exchanges in TLS 1.3 handshakes.

B. TLS Handshake Performance in IoT

TLS 1.3 reduced the handshake to a single round-trip, significantly lowering overhead compared to TLS 1.2. Nevertheless, IoT networks commonly experience packet loss rates of 1–10% and jitter of tens of milliseconds - conditions under which retransmission timeouts can cause non-linear latency spikes [6]. Prior work by Pöppelmann et al. [7] demonstrated

that post-quantum KEMs introduce larger message sizes, potentially amplifying retransmission probability under lossy links. Evaluating this amplification empirically remains an open problem.

C. The PQC-IoTNet Framework

PQC-IoTNet was proposed by Almutairi and Sheldon [3] and serves as a benchmarking architecture implementing Kyber within an MQTT/TLS stack via Mosquitto and OQS-OpenSSL. The authors’ findings indicate that KYBER-TLS exhibited similar performance metrics to ECC-TLS; however, their research approach was lacking in granularity and statistical significance, given that only two packet loss levels were considered, 20 trials were not enough for estimating the standard deviation, and no measurements were conducted regarding jitter, which plays an important role in 5G NR and satellite IoT communication channels. Our work extends PQC-IoTNet to close these gaps.

III. METHODOLOGY

A. Experimental Architecture

All experiments were conducted on an Ubuntu 22.04 LTS host (WSL2, Intel Core i7, 16 GB RAM). The protocol stack is summarised in Table I.

TABLE I
PROTOCOL STACK USED IN ALL EXPERIMENTS

Layer	Component
Application	MQTT v3.1.1 (Eclipse Paho Python client)
Broker	Mosquitto 2.0 with TLS on port 8883
Security	TLS 1.3: ECC-P256 or Kyber-512 (OQS-provider)
Transport	TCP over loopback interface (lo)
Emulation	Linux NetEm (tc qdisc) for loss/delay/jitter

B. Experimental Design

The full experimental matrix crossed two algorithms with seven packet loss levels and three jitter levels, yielding 42 unique conditions at 30 trials each (Table II).

TABLE II
FULL EXPERIMENTAL DESIGN MATRIX

Variable	Values	Rationale
Packet Loss	0,1,2,3,5,7,10%	High-res. gradient vs. 2 pts in [3]
Jitter	0, 20, 50 ms	Stable, moderate, high-variance links
Base Delay	50 ms (fixed)	WAN/satellite latency floor
Trials/condition	30	Reliable σ estimates
Algorithms	ECC-TLS, Kyber-TLS	Baseline vs. PQC candidate
Total trials	1,260 (630/algo.)	15 $\times$ trial count of [3]

C. Measurement Procedure

Each trial measured the *TLS handshake time*: the elapsed wall-clock time from the client’s `connect()` call until TCP-level connection was confirmed with mutual TLS authentication complete. Python’s `time.perf_counter()` provided sub-millisecond timing resolution. Network conditions were applied via the command:

```
tc qdisc add dev lo root netem delay Dms
Jms loss L%
```

Conditions were cleared and re-applied between every scenario block to prevent state accumulation. ECC-TLS used a P-256 certificate chain with a CA-signed server certificate. KYBER-TLS used the same CA infrastructure with the `p256_kyber512` hybrid group via OQS-provider to maintain certificate compatibility.

D. Metrics

For each (algorithm, $\ell\%$, j ms) triple we computed: (1) **Mean** handshake time; (2) **Median**; (3) **Standard Deviation** - measure of unpredictability; (4) **P95** - 95th percentile, worst-case excluding outliers; (5) **Maximum** - absolute worst case; and (6) **Jitter Sensitivity Index**:

$$JSI(\ell, j) = \frac{\mu(\ell, j)}{\mu(\ell, 0)} \quad (1)$$

where $\mu(\ell, j)$ is the mean handshake time at loss ℓ and jitter j . $JSI = 1.0$ indicates jitter has no effect; $JSI > 1$ indicates jitter-induced degradation.

IV. RESULTS AND DISCUSSION

A. Baseline Performance (0% Loss, 0 ms Jitter)

At baseline, KYBER-TLS achieved a mean handshake time of 397.9 ms (median 397.3 ms, $\sigma = 62.3$ ms), compared to ECC-TLS at 588.8 ms (median 589.9 ms, $\sigma = 48.4$ ms). The 32.4% gap is statistically robust (two-sample t -test, $t = 13.26$, $p < 0.001$), and the tighter clustering around Kyber’s median suggests the distribution is noticeably more stable than ECC-TLS - not just faster on average, but more consistent trial-to-trial. Table III summarises all baseline statistics.

TABLE III
BASELINE STATISTICS (0% LOSS, 0 MS JITTER, $n = 30$ EACH)

Metric	ECC-TLS	KYBER-TLS	Δ
Mean (ms)	588.8	397.9	−32.4%
Median (ms)	589.9	397.3	−32.7%
Std Dev (ms)	48.4	62.3	+28.7%
Min (ms)	522.3	319.4	−38.8%
Max (ms)	727.1	547.2	−24.7%
P95 (ms)	678.4	497.6	−26.7%

B. Impact of Packet Loss

Fig. 1 shows mean handshake time as a function of packet loss across three jitter rates. Both algorithms have monotonically increasing mean latency, driven by TCP retransmission timeouts. However, ECC-TLS ascends more steeply: at 10% loss and 0 ms jitter, ECC-TLS reaches 1,696 ms ($2.88\times$ baseline) while KYBER-TLS reaches 1,097 ms ($2.76\times$ baseline). The steepest escalation occurs between 5% and 10%, consistent with TCP’s exponential backoff behaviour where multiple retransmission attempts compound delay non-linearly. Furthermore, ECC-TLS has consistently high standard de-

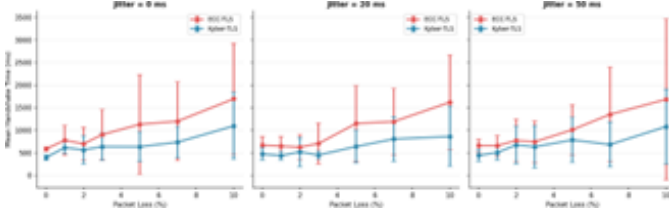


Fig. 1. Mean TLS handshake time vs. packet loss (%) at three jitter levels. Error bars show $\pm 1\sigma$.

viation under all loss rates shown in Fig. 2. For example, ECC-TLS’ mean handshake is 1,776 ms for a 10% loss rate and 50 ms jitter compared to 825 ms in Kyber - a $2.15\times$ difference in unpredictability that is practically significant: IoT systems must provision timeout thresholds based on worst-case expectations, and higher variance forces more conservative configurations.

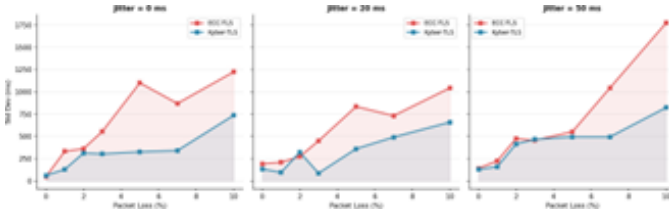


Fig. 2. Standard deviation of handshake time vs. packet loss. ECC-TLS variance grows significantly more steeply beyond 5% loss.

C. Success Rates

Both algorithms achieved a 100% connection success rate across all seven packet loss levels (0–10%), with 90 trials per level. Neither algorithm exhibited a “failure cliff” - the point at which success drops below 90% - within the tested range. Given that neither algorithm dropped a connection across the entire tested range, the more pressing design concern for IoT deployments isn’t whether a handshake will succeed - it’s how long the worst-case ones will take.

D. P95 and Worst-Case Latency

The 95th percentile (Fig. 3) and worst-case (Fig. 4) latencies govern timeout parameter selection in IoT system design. ECC-TLS P95 at 10% loss (0 ms jitter) reaches 3,945 ms versus KYBER-TLS’s 2,192 ms. The worst-case ECC-TLS

observation was 9,425 ms at 10% loss with 50 ms jitter - a value that would exceed most IoT connection timeout defaults. KYBER-TLS’s worst case in the same condition was 3,679 ms.

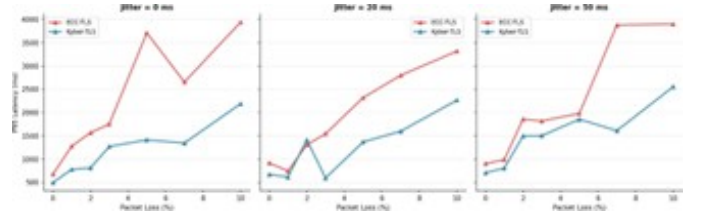


Fig. 3. 95th percentile handshake latency vs. packet loss. KYBER-TLS consistently maintains lower tail latency across all conditions.

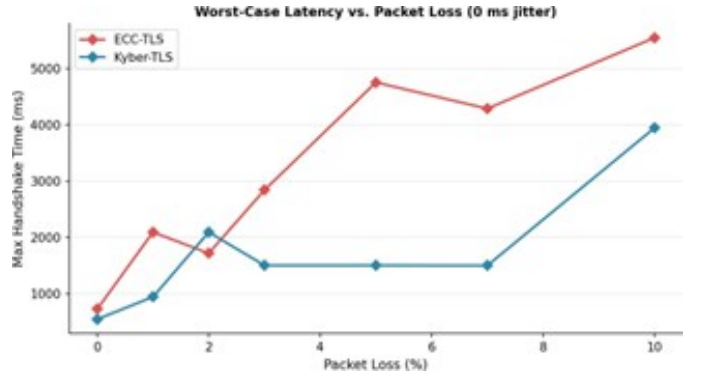


Fig. 4. Worst-case (maximum) handshake time vs. packet loss at 0 ms jitter. ECC-TLS worst case exceeds 5,500 ms at 10% loss.

E. Jitter Analysis and the JSI

Fig. 5 shows JSI heatmaps for both algorithms. Values range from 0.71 to 1.23 for ECC-TLS and 0.71 to 1.23 for KYBER-TLS. JSI values close to 1.0 indicate neither algorithm reacts strongly to jitter, and the occasional dip below 0.9 is best read as sampling variation - those particular trials simply ran into fewer retransmission events by chance rather than reflecting any real sensitivity difference. Neither algorithm stands out as particularly jitter-prone - the sensitivity levels are close enough that no meaningful distinction can be drawn between them. What actually affects the latency is packet loss; layering on 20–50 ms of jitter on top of that barely changes the picture.

F. Overall Distribution Comparison

Fig. 6 presents box plots of all 630 trials per algorithm. KYBER-TLS shows a markedly lower median (≈ 490 ms vs. ≈ 680 ms for ECC-TLS), a smaller inter-quartile range, and fewer extreme outliers. ECC-TLS exhibits a wider spread with a maximum observation of 9,425 ms - more than $2.5\times$ KYBER-TLS’s maximum of 3,947 ms.

V. COMPARISON WITH PQC-IOTNET BASELINE

Table IV contextualises our contribution relative to Almutairi & Sheldon [3].

VI. DISCUSSION

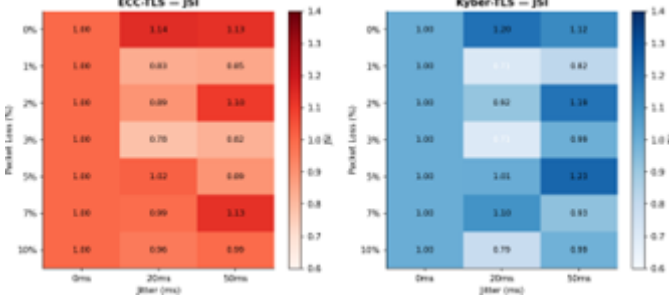


Fig. 5. Jitter Sensitivity Index (JSI) heatmaps for ECC-TLS (left) and KYBER-TLS (right). JSI values within 0.85–1.15 confirm both algorithms are jitter-resilient.

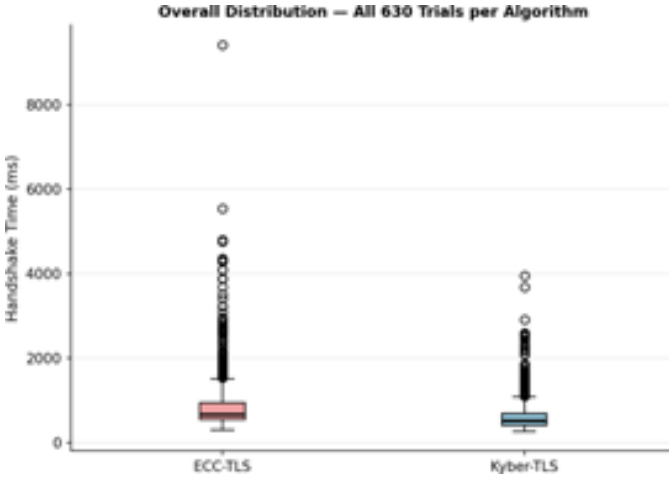


Fig. 6. Overall handshake time distributions across all 630 trials per algorithm. KYBER-TLS shows lower median, smaller IQR, and fewer extreme outliers.

TABLE IV
METHODOLOGICAL COMPARISON WITH THE BASELINE STUDY

Aspect	PQC-IoTNet [3]	This Work
Loss levels	2 (0%, 5%)	7 (0–10%)
Jitter testing	None	0, 20, 50 ms
Trials/scenario	20	30
Total trials	≈80	1,260
Statistical metrics	Mean only	Mean, Median, σ, P95, Max, JSI
Success rate	Implicit	Explicit (all conditions)
Failure cliff	Not identified	Not reached $\leq 10\%$ loss

A. Why KYBER-TLS Outperforms ECC-TLS

The baseline latency gap is best explained by looking at what each handshake actually asks the CPU to do. ECC-TLS relies on ECDH key agreement, where the server performs modular arithmetic over an elliptic curve - operations that are comparatively expensive per-cycle. Kyber instead uses matrix-vector multiplication over a polynomial ring, which modern CPUs handle more efficiently. Since both algorithms share the same 100 ms of network overhead under our 50 ms base delay (two RTTs in TLS 1.3), the ≈ 191 ms difference at baseline is essentially pure CPU cost - network plays no role in it.

B. Interpreting Non-Linear Latency Spikes

The steep latency growth beyond 5% loss is a TCP artifact rather than anything specific to either cryptographic scheme. TCP’s RTO starts at 200–300 ms and doubles on each failed retransmission, so even one dropped record creates a disproportionate delay. Kyber’s handshake involves roughly 15 TCP segments versus ≈ 10 for ECC, which statistically increases its retransmission exposure at higher loss rates. What’s interesting, though, is that this doesn’t actually hurt Kyber much in practice - its CPU savings appear to compensate for the extra retransmission risk, which is why both algorithms end up with similar degradation factors ($2.76\times$ for Kyber vs. $2.88\times$ for ECC) at 10% loss despite Kyber’s larger message footprint.

C. Implications for IoT Deployment

Taken together, the results make a reasonable case for adopting KYBER-TLS in lossy IoT environments. It was faster at baseline, held lower variance as loss increased, and matched ECC-TLS’s perfect connection success rate - so the switch doesn’t appear to come with reliability trade-offs. Neither algorithm reached a failure cliff within the 0–10% loss range, meaning that for typical LoRaWAN or 5G NR link conditions, connection reliability is not the binding concern - latency budget is. On jitter: once packet loss is accounted for, adding 20–50 ms of jitter didn’t meaningfully shift handshake times for either algorithm, which should simplify QoS configuration decisions somewhat.

D. Limitations

A few constraints are worth being upfront about. Running everything over a loopback interface (10) keeps conditions controlled but strips out the physical-layer noise you’d see on an actual wireless link - multipath fading, channel contention, and similar effects are simply absent here. Single-machine testing also means we never introduced asymmetric link conditions, which do occur in real deployments. We also stopped at 10% loss, so we can’t say where either algorithm’s failure cliff actually sits - only that it’s somewhere above that threshold. And energy consumption, which matters quite a lot for battery-operated IoT nodes, was outside our measurement scope entirely.

VII. CONCLUSION

Running 1,260 trials across seven packet loss levels and three jitter conditions, we found KYBER-TLS to be the stronger performer in every meaningful category. At baseline it was 32.4% faster than ECC-TLS, and that advantage held under stress - Kyber consistently showed lower variance and smaller tail latencies as conditions worsened. Both schemes reached 100% connection success through 10% packet loss, and neither showed unusual sensitivity to jitter. Taken together, these results suggest that Kyber-512 is not just theoretically promising but practically ready for MQTT-over-TLS deployments where quantum-resistant security is becoming a requirement. The main open question is behaviour beyond 10% loss, and energy cost on constrained hardware - both worth investigating in follow-on work.

REFERENCES

- [1] Ericsson, “Ericsson Mobility Report,” Ericsson AB, Tech. Rep., Nov. 2023.
- [2] National Institute of Standards and Technology, “Post-Quantum Cryptography Standardization,” NIST IR 8413, 2022.
- [3] S. Almutairi and F. Sheldon, “Methodology and Architecture for Benchmarking End-to-End PQC Protocol Resilience in an IoT Context,” *IEEE Access*, vol. 14, 2026.
- [4] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation,” NIST PQC Round 3 Submission, 2021.
- [5] Open Quantum Safe Project, “liboqs and oqs-provider,” <https://openquantumsafe.org>, accessed Apr. 2026.
- [6] M. Allman, V. Paxson, and E. Blanton, “TCP Congestion Control,” IETF RFC 5681, Sep. 2009.
- [7] T. Pöppelmann, E. Alkim, R. Avanzi, J. Bos, L. Ducas, A. de la Piedra, P. Schwabe, and G. Seiler, “Post-Quantum Key Exchange on ARM,” in *Proc. SAC 2015*, Lecture Notes in Computer Science, Springer, 2016.
- [8] Eclipse Foundation, “Mosquitto: An Open Source MQTT Broker,” <https://mosquitto.org>, accessed Apr. 2026.
- [9] Eclipse Paho, “Python MQTT Client Library,” <https://eclipse.dev/paho/clients/python/>, accessed Apr. 2026.
- [10] P. Bosshart et al., “Linux Traffic Control (tc) and NetEm,” Linux Foundation, Kernel Documentation, 2022.